FRED TALKS

26th October 2026

CONTENTS

Astra for Coding: Why Are We Doing This Again?
ARMIN RONACHER · 4171 WORDS

Astra for Coding: Why Are We Doing This Again?

ARMIN RONACHER · 09 SEP 2026 · SOURCE

I'm more and more convinced that all of AI engineering is Neijuan (　, meaning curl inwards). In China it describes a system that demands ever more effort and competition without improving output. The way in which it sometimes shows up in the West is the 996 nonsense. The English term for Neijuan is "Involution" from the book Agricultural Involution. Agricultural involution describes the intensification of farming that raises productivity per square meter while leaving productivity per head unchanged.

That's how I feel about AI right now.

Which brings me to GPT 6 Astra. Astra is by all accounts an incredibly impressive model. There is really not much I can say against this. It's amazing at computer use, understands images and complex topics, and it's relentless in its pursuit of completion. It is absolutely impressive; these types of models are going to change the world in one form or another.

But at least for the moment I don't know how to work with it for actual software engineering. Since that got quite a bit of attention on Twitter, I figured I might summarize my thoughts and just share what kind of code comes out of this thing.

My Slop Factory

"Armin, you should run a software factory!" I've heard that a few times now, so I figured I might celebrate the release of it by running a little software factory over the weekend. If everybody builds slop 3D games, then I should do something useful with it. My software factory was intentionally set up to let the model decide the how of the workflow entirely. It was free to manage its own context and could maintain its own records in an `agent-notes` folder. Then it spun off subagents to work on stuff. The goal? What if we had a Python with virtual threads and lexical scoping. And well, I burned a full reset's worth of ChatGPT tokens on this which appears to be around 4 billion tokens. 35 hours later, the factory has delivered absolutely nothing of value and also not taught me anything about how to operate a better one.

Disposable Code vs Committed Code

In a world where code for tool calls is optimized for token efficiency and "getting the job done", I wonder if there is really enough signal going to the training processes for "a human understands what is going on". I would say that quite a lot of the code I get out of Astra is in my mind "objectively bad". But it's objectively bad by my human sense. Maybe it's objectively good for a codebase that is entirely written by agents and only needs to be understood by agents.

Which is why I'm honestly asking myself more and more why we are doing this. These new models are absolutely amazing, for sure. But I'm more and more skeptical that the trajectory they are on still lends itself to present-day software engineering processes. The reason why I'm asking why we are doing this is because I felt like we achieved a pretty good spot for software engineering with those models, and that is the part of the AI economy where it was possible to show a positive return. But for how much more Fable costs, for how much more Astra costs, I do not feel like the results are there.

In fact, with Astra and Fable I feel like not only are the costs astronomical, but the models are also just not for me as a software engineer. And presumably that's because these models increasingly are for other people. For lawyers, 3D artists, mathematicians, whoever uses computer use, etc.

And potentially as a byproduct of enabling all of this, you can now slop your way to a one-shot 3D game over the weekend which looks impressive. And probably you can now run a software factory for as long as you don't care about the code.

I'm sure I will get used to this, but man this stuff is weird.

Postscriptum: speaking of weird: how is it that these models, in a sandbox, with supposedly no way to communicate with other agents, manage to find the same public wikis as a scratch pad for agent communication? Did they collude during training runs to remember resources on the internet which might come in handy in the future?

1. I should clarify that I have done experiments like this before. Typically they do not run this long and the agent leaves behind a maybe imperfect but still digestible piece of software.↩

But it produced a lot of code and input prompts, and so there is stuff I was able to study. And well, it shows behavior that I'm not used to with Sol and earlier OpenAI models [1]. I have since encountered the same issues with regular programming with Astra, so it's not a result of just the factory.

I think I'm suspecting something is going "wrong" in the training process. The model is greatly rewarded for succeeding on long-horizon tasks, but presumably there is very little punishing going on for "shitty code." The apparent result is that Astra is amazing at producing 3D stuff and it can keep going for a very long time, coming up with its own work in the process. I had it do quite a bit of reverse engineering of my robot vacuum in ways that were quite impressive. So it's definitely cool!

Codegolf Tool Calls

The first issue I have with Astra comes from the type of code that it uses for tool calls. Codex increasingly has been relying on "just bash" to do more and more operations. For a few versions now the original Codex harness just uses `sed` and other tools to read files. You just usually can't see them because Codex parses the bash commands and hides them if it recognizes them. But Astra … really loves Python? That is not much of a surprise because even older OpenAI models had a tendency to sometimes use on-demand Python code to read and manipulate files at times, but Astra does it really quite excessively for me.

Now here is an important disclaimer: this project is *very meta* here because I worked *on* the CPython interpreter. But I can assure you that I have seen this model do weird Python things even in TypeScript code in Pi. But I have the most evidence of odd code from when I had the thing work over the weekend with *zero* oversight from my slop factory.

That it writes Python is not interesting; the type of Python is interesting, and I collected some outputs for you to gloss over.

Python string splicing to edit C code
In the Codex harness I found multiple cases where subagents resorted fully to manual string manipulation with Python instead of using the patch tool.

```
python3 - <<'PY'
from pathlib import Path
p=Path('Include/internal/pycore_intrinsics.h');s=p.read_text().replac
p=Path('Python/intrinsics.c');s=p.read_text();idx=s.index('#define IN
   capture. A replaced cell's finalizer may reenter module __annotate
static PyObject *
```

```
retain_annotation_cells(PyThreadState *tstate, PyObject *holders)
{
    if (!PyTuple_CheckExact(holders)) {
        PyErr_SetString(PyExc_TypeError, "annotation holders must be
        return NULL;
    }
    Py_ssize_t size = PyTuple_GET_SIZE(holders);
    PyObject *previous = PyTuple_New(size);
    if (previous == NULL) return NULL;
    for (Py_ssize_t i = 0; i < size; i++) {
        PyObject *holder = PyTuple_GET_ITEM(holders, i);
        if (!PyCell_Check(holder)) {
            Py_DECREF(previous);
            PyErr_SetString(PyExc_TypeError, "annotation holder must
            return NULL;
        }
        PyObject *cell = PyCell_Get(holder);
        PyTuple_SET_ITEM(previous, i, cell == NULL ? Py_NewRef(Py_Non
    }
    return previous;
}

''' +s[idx:];s=s.replace('    INTRINSIC_FUNC_ENTRY(INTRINSIC_AWAIT_BL
p=Path('Python/codegen.c');s=p.read_text();idx=s.index('static int\nc
codegen_retain_annotation_cells(compiler *c, location loc, PyObject *
{
    Py_ssize_t pos = 0;
    PyObject *binding, *holder;
    while (PyDict_Next(captures, &pos, &binding, &holder)) {
        ADDOP_NAME(c, loc, LOAD_CLOSURE, holder, cellvars);
    }
    ADDOP_I(c, loc, BUILD_TUPLE, PyDict_GET_SIZE(captures));
    ADDOP_I(c, loc, CALL_INTRINSIC_1, INTRINSIC_RETAIN_ANNOTATION_CEL
    return SUCCESS;
}

''' +s[idx:]
a=s.index('                if (conditional_annotation_index != NULL)
b=s.index('                if (captures != NULL) {',a)
                    PyObject_RichCompareBool(PyTuple_GET_ITEM(old, 2), po
        }
        if (PyList_Append(result, token) < 0) { Py_DECREF(token); got
        Py_DECREF(token);
    }
    for (; index < PyList_GET_SIZE(it->pending); index++) {
        if (PyList_Append(result, PyList_GET_ITEM(it->pending, index)
    }
    Py_SETREF(it->pending, result);
    Py_DECREF(events);
    return 0;
error:
    Py_DECREF(events);
    Py_DECREF(result);
    return -1;
}
```

The failure case here seems somewhat obvious: the model is trained for token efficiency for tool calling which also looks like code, and sometimes it seems to be taking that code into a place where it should not be: the codebase.

35 Hours on a Single Prompt

I'm not really sure what to say here, but the slop machine was running for 35 hours until I turned it off. In that time it produced a net addition of 75k lines of code and it did not stop. In the 35 hours it burned around 1B tokens for a total of around 1200 USD in raw API costs. It managed to produce 79 commits, and that comes to a cost of around 15.5 USD per commit, and the agents exchanged around 1400 messages.

I honestly do not need an agent to run for 35 hours on a single prompt. It clearly does not work or result in reasonable outputs.

So obviously: prompting it like this is stupid. But when left unattended, it *will* keep going, and earlier models did not do that. Even Fable wasn't as crazy as that. When you accidentally give it slightly too big of a task, it will continue until it succeeds, even if it burns through an entire subscription.

And that's more or less why right now I do not manage to trust this model much. It has shown that it will commit slop, and it requires me to review it more as a result. Even if the failure rate is quite low, I would not want this.

This is not the codebase's coding style, and quite frankly it should not be anyone's coding style. I do not understand what motivated the model to do this.

```
static int
apply_layout(tokenizeriterobject *it)
{
    PyObject *source = PyBytes_FromStringAndSize(it->tok->source.byte
    if (source == NULL) return -1;
    PyObject *events = _PyPegen_tokenize_layout(PyBytes_AS_STRING(sou
    Py_DECREF(source);
    if (events == NULL) return -1;
    PyObject *result = PyList_New(0);
    if (result == NULL) { Py_DECREF(events); return -1; }
    Py_ssize_t index = 0;
    PyObject *first_pos = PyTuple_GET_ITEM(PyList_GET_ITEM(it->pendin
    PyObject *last_pos = PyTuple_GET_ITEM(PyList_GET_ITEM(it->pending
    PyObject *previous = NULL;
    for (Py_ssize_t i = 0; i < PyList_GET_SIZE(events); i++) {
        PyObject *event = PyList_GET_ITEM(events, i);
        if (previous && PyObject_RichCompareBool(previous, event, Py_
        previous = event;
        PyObject *token = layout_token(it, event);
        if (token == NULL) goto error;
        if (token == Py_None) { Py_DECREF(token); continue; }
        PyObject *pos = PyTuple_GET_ITEM(token, 2);
        if (PyObject_RichCompareBool(pos, first_pos, Py_LE) == 1 ||
            PyObject_RichCompareBool(pos, last_pos, Py_GT) == 1) { Py
        while (index < PyList_GET_SIZE(it->pending)) {
            PyObject *old = PyList_GET_ITEM(it->pending, index);
            int cmp = PyObject_RichCompareBool(PyTuple_GET_ITEM(old,
            if (cmp < 0) { Py_DECREF(token); goto error; }
            if (!cmp) break;
            if (PyList_Append(result, old) < 0) { Py_DECREF(token); g
            index++;
        }
        if (index < PyList_GET_SIZE(it->pending)) {
            PyObject *old = PyList_GET_ITEM(it->pending, index);
            long kind = PyLong_AsLong(PyTuple_GET_ITEM(old, 0));
            if ((kind == NL || kind == NEWLINE || kind == INDENT || k
```

```
# Move lookup before conditional registration and retain old cells be
lookupstart=s.index('                PyObject *captures = _PyCompile_
lookup=s[lookupstart:b].replace('                    return ERROR;','
s=s[:lookupstart]+s[b:]
setup=lookup+'''                if (captures != NULL && codegen_retai
                    Py_XDECREF(conditional_annotation_index); return
                }
'''
s=s[:a]+setup+s[a:]
needle='                        ADDOP_NAME(c, loc, STORE_DEREF, holde
s=s.replace(needle,'                            ADDOP_NAME(c, loc, STORE_
p=Path('Include/internal/pycore_magic_number.h');s=p.read_text().repl
p=Path('Lib/test/test_block_annotation_captures.py');s=p.read_text();
        module=execute("""\\
            events=[]
            class V:
                def __init__(self,n): self.n=n
                def __del__(self):
                    if self.n == 0: events.append(__annotate__(1))
            for i in range(2):
                x=V(i) # bind x y
                y=i
                value:(x.n,y)
        """)
        self.assertEqual(module.events,[{'value':(1,1)}])
        self.assertEqual(module.__annotate__(1),{'value':(1,1)})

''' +s[idx:];p.write_text(s)
PY
make -j1 > /tmp/block-annotations-build7.log 2>&1
```

Socket codegolf

In the middle of a conversation the agent ran into "Bad file descriptor" on a test and Astra decided it needs to see if file descriptors can be passed over Unix sockets on macOS in a super compressed manner:

```
/usr/bin/python3 - <<'PY'
import socket,os,array
for into in (False,True):
 a,b=socket.socketpair();fd=os.open(os.devnull,os.O_RDONLY);b.sendmsg
```

```
 if into:r=a.recvmsg_into([bytearray(1),bytearray(),bytearray(19)],so
 else:r=a.recvmsg(20,socket.CMSG_SPACE(4),socket.MSG_PEEK|socket.MSG_
 print('peek',r,flush=True)
 rights=array.array('i',r[1][0][2]);print('rights',rights,flush=True)
 for f in rights:
  try: print('stat',os.fstat(f))
  except Exception as e: print('error',e)
 r=a.recvmsg(20,socket.CMSG_SPACE(4),socket.MSG_DONTWAIT);print('cons
 a.close();b.close();os.close(fd)
PY
```

Python for agent notes patching

The agent notes were rather consistently updated with Python:

```
python3 - <<'PY'
from pathlib import Path
p=Path('agent-notes/live/block-with-bindings.md');s=p.read_text().rep
# Keep the final evidence readable without rewriting historical paren
for a,b in [('all328','all 328'),('pass31','pass 31'),('pass all328',
s += '\nKey source review: Python/symtable.c:603 (discovery), :3985 (
p.write_text(s)
for name in ('STATE.md','build-and-test.md'):
 p=Path('agent-notes/live')/name;s=p.read_text()
 for a,b in [('build:117','build: 117'),('paths.18','paths. 18'),('pa
 p.write_text(s)
PY
git diff --check
git add -u
git add Lib/test/test_block_with_bindings.py agent-notes/done/asyncio
git diff --cached --stat
git commit -m 'Add explicit with and async with header bindings'
```

Using Python to run Node.js

In multiple cases it used Python to spawn Node.js on another machine. It first wrote the script, then it used Bash to run Python, then that program ran Node.js via `prlctl` on my Windows box.

```
import subprocess
code = "const{readFileSync}=require('fs');const{strict:a}=require('as
subprocess.run(['prlctl', 'exec', 'Windows 11', '--current-user', 'C:
```

This code style does not exist in the CPython code base, yet it shows up in newly generated code.

```
PyObject *info = PyTuple_Pack(3, name, mangled, suite->su_id);
PyObject *flags = PyLong_FromLong(DEF_LOCAL);
if (key == NULL || info == NULL || flags == NULL ||
    PyDict_SetItem(suite->su_bindings, mangled, key) < 0 ||
    PyDict_SetItem(st->st_cur->ste_block_bindings, key, info) < 0 ||
    (private && PyDict_SetItem(st->st_binding_info, key, info) < 0) |
    (private && PyDict_SetItem(st->st_cur->ste_symbols, key, flags) <
    Py_DECREF(mangled); Py_XDECREF(key); Py_XDECREF(info); Py_XDECREF
    goto error;
}
Py_DECREF(mangled); Py_DECREF(key); Py_DECREF(info); Py_DECREF(flags)
```

Random indexes in production code

As with the numbers for the operators, it also uses random integers in a list to stash away state.

```
def _register_task(task):
    """Register an asyncio Task scheduled to run on an event loop."""
    _scheduled_tasks.add(task)
    if _task_accelerator is not None:
        _task_accelerator[6](task)


def _register_eager_task(task):
    """Register an asyncio Task about to be eagerly executed."""
    _eager_tasks.add(task)
    if _task_accelerator is not None:
        _task_accelerator[8](task)


def _enter_task(loop, task):
    if (_task_accelerator is not None and
            _task_accelerator[5]() is loop and loop not in _current_t
        return _task_accelerator[1](loop, task)
    # ...
```

Hideous tokenizer code in C

```
int saved_errno = errno;
PyObject *result = NULL;
Py_ssize_t value;
/* No intervening Python frame: these exercise ambient C provenan
switch (operation) {
    case 0: result = PyObject_CallNoArgs(callback); break;
    case 1: result = PyNumber_Add(callback, other); break;
    case 2: result = PyNumber_Negative(callback); break;
    case 3: result = PyObject_RichCompare(callback, other, Py_LT)
    case 4:
        value = PyObject_IsTrue(callback);
        if (value >= 0) result = PyBool_FromLong(value);
        break;
    case 5:
        value = PyObject_Length(callback);
        if (value >= 0) result = PyLong_FromSsize_t(value);
        break;
    case 6: result = PyObject_GetIter(callback); break;
    case 7: result = PyIter_Next(callback); break;
    case 8: result = PyObject_GetItem(callback, other); break;
    /* ... */
    case 21:
        result = PyType_Type.tp_call(callback, other, NULL);
        break;
    case 22: case 23: case 24: case 25: case 26:
        result = conversion_probe(operation, callback); break;
    case 27: case 28: case 29:
        result = protocol_probe(operation, callback, other); brea
    case 30: case 31: case 32: case 33: case 34: case 35:
    case 36: case 37: case 38: case 39: case 40: case 41:
    case 42: case 43: case 44: case 45: case 46:
    case 47: case 48: case 49: case 50: case 51: case 52:
    case 53: case 54: case 55: case 56: case 57: case 58: case 59
    case 60: case 61: case 62: case 63: case 64: case 65: case 66
    case 67: case 68: case 69: case 70: case 71: case 72:
        result = collection_probe(operation, callback, other); br
    default: PyErr_SetString(PyExc_ValueError, "bad probe operati
}
```

Multiple same-line macro invocations in C

Python to run Node.js to run PowerShell

Since it was already doing that, it used Bash to run Python to then run Node.js to then use Node.js to invoke PowerShell.

```
import subprocess
code = "process.env.PSModulePath='C:/Windows/System32/WindowsPowerShe
subprocess.run(['prlctl', 'exec', 'Windows 11', '--current-user', 'C:
```

You can consider this amusing, but I have some questions here. The first problem with this is that it’s unreadable for a human. If you wanna follow along with what is going on, then good luck. Particularly once it opts out of using the edit tools that the harness provides, you’re going to have to resort to using the diff viewer of the final artifacts since it’s almost impossible to visualize the changes as they happen by reading the code.

This is not quite as bad in Pi for the most part because I mostly see it editing with the `edit` tool. When however goes all bananza with subagents (where the agent believes nobody is looking) it’s resorting to all kinds of increasingly bizarre behavior. I actually don’t know if the model thinks someone is looking, but that’s the vibe I’m getting.

But then it starts doing the same nonsense in code that actually gets committed. I have mostly seen this in tests, but you can also see this for instance when it writes JavaScript or CSS embedded in HTML. It almost seems like when it’s “one step removed” from regular code, it starts falling into these patterns.

Here are some unit tests that it created:

Complete disregard for whitespace and indentation

```
def test_unpack_suspension_and_continuation_close(self):
    from continuations import Continuation,suspend
    readers=[]
    class Source:
        def __iter__(self):
            yield 1
            suspend('unpacking')
            yield 2
    ns=execute('''
        def run():
            a,b='old-a','old-b'
            readers.append(lambda: (a,b))
            def a,b=Source()
```

```
            suspend('published')
    ''',Source=Source,readers=readers,suspend=suspend)
    with Continuation(ns['run']) as continuation:
        self.assertEqual(continuation.resume(),'unpacking')
        self.assertEqual(readers[0](),('old-a','old-b'))
        self.assertEqual(continuation.resume(),'published')
        self.assertEqual(readers[0](),(1,2))
    class Value:pass
    refs=[];frames=[];callbacks=[]
    ns=execute('''
        def run():
            for def x in [Value()]:
                refs.append(weakref.ref(x))
                frames.append(sys._getframe())
                callbacks.append(lambda: x)
                suspend('body')
    ''',Value=Value,refs=refs,frames=frames,callbacks=callbacks,weakr
    with Continuation(ns['run']) as continuation:self.assertEqual(con
    self.assertNotIn('x',frames[0].f_locals)
    self.assertIsNotNone(refs[0]());callbacks.clear();self.assertIsNo

def test_ast_roundtrips_and_future_annotation_unparse(self):
    source='callback=lambda {for def a, [b,*rest] in [(1,[2,3])] {ret
    tree=ast.parse(source);node=tree.body[0].value.body[0]
    self.assertIsInstance(node,ast.ForBinding)
    self.assertEqual(node._fields,('target','iter','body','orelse','t
    self.assertEqual(node.lineno,1);self.assertGreater(node.end_col_o
    self.assertEqual(ast.dump(tree),ast.dump(ast.parse(ast.unparse(tr
    ns=execute('from __future__ import annotations\ndef f(arg: '+sour
    self.assertEqual(eval(ns['f'].__annotations__['arg'])(),(1,2,[3])
    tree=ast.parse('async def f():\n async for def x in values: pass
    self.assertIsInstance(tree.body[0].body[0],ast.AsyncForBinding)
    self.assertEqual(ast.dump(tree),ast.dump(ast.parse(ast.unparse(tr
```

So at least in some situations, the Python slop that it normally code-golfs for token-efficient tool calls leaks into the Python code it generates that should be stored. And well, it's clearly more token efficient. The two unit tests above, when indented to the class structure they were in, are 10% more token efficient in this form than after a `ruff format`.

It's AGI If You Don't Look

I think there are a handful of things happening now that are pushing the whole thing in directions that are in conflict with one another. The training runs for these models are rapidly accelerating and they are now presumably also moving towards recursive self-improvement. The reward for the models is probably a combination of token efficiency, task completion rate and maybe some simple indicators like cyclomatic complexity. But we humans don't think of code that is readable or understandable by simple, readily quantifiable metrics. All those things you can easily measure in isolation, and you can also optimize for them quite locally.

But these local optimizations do not produce global optimums, and the fewer of us are looking at the output, the less it matters. Obviously my software factory ran aground over the ~35 hours that it ran, but you can see the gradual regression towards insanity from the notes that it produced. For instance the task naming in the task file starts with an optimistic 1, 2, 3, 5, 5a but then eventually gets to 8a, 8a1, and then ends up with 8b2c2b3 and "8b2c2b2b checkpoint1". The code that it produced got ever more wild. I don't want to bore you with what it tried to build, but here are some example pieces of the interpreter changes:

Hardcoded constants everywhere
I have no idea where it got those numbers from, but at one point it started passing random constants from one module to a C implementation. Initially that started out as a function that it mainly needed to do test assertions, but just before I turned off that experiment, that function started to be relied upon by non-test code as well.

```
static PyObject *
native_probe_run_impl(PyObject *callback, int sleep, int operation, P
{
    pthread_mutexattr_t attr;
    pthread_mutex_t mutex;
    pthread_mutexattr_init(&attr);
    pthread_mutexattr_settype(&attr, PTHREAD_MUTEX_RECURSIVE);
    pthread_mutex_init(&mutex, &attr);
    pthread_mutexattr_destroy(&attr);
    pthread_mutex_lock(&mutex);
    int previous = native_sentinel;
    pthread_mutex_t *previous_mutex = native_mutex;
    native_sentinel = previous + 1;
    native_mutex = &mutex;
    PyThreadState *tstate = PyThreadState_Get();
    PyGILState_STATE gil = PyGILState_Ensure();
```